CSC 255

Unit 8 Programming Assignment 1 [65 points]

Mazes

An array-based disjoint set class is a data structure used to efficiently represent disjoint sets, where no element is shared between sets. Use the disjoint set data structure with Kruskal's minimum spanning tree algorithm to generate a maze of size 20x20. It's your choice on how to display the maze, using graphics or text-based output, but the maze should resemble **Figure 1**. Write the resulting maze generated to a file named *maze.maz*. **[30 points]**

```
+-+-+-+-+
|S|  |    |
+ + + +-+
|    | | |
+ +-+ + +
|        |
+ +-+ + +
| |    |E|
+-+-+-+-+
```

Figure 1

Then use a breadth-first or depth-first search to solve the maze generated. A solved maze might look something like **Figure 2**. Note, depending on the search used, the number of paths traversed to find a solution will vary. Write the solved maze to a file named *maze_solved.maz*. **[30 points]**

```
+-+-+-+-+
|S|  |    |
+~+ + +-+
|~   | | |
+~+-+ + +
|~~~~~~~~|
+ +-+ +~+
| |    |E|
+-+-+-+-+
```

Figure 2

Kruskal's algorithm

- Kruskal's algorithm is a minimum spanning tree algorithm that's commonly used for maze generation. It starts with each cell as a separate set and iteratively selects the shortest edge that connects two different cells/sets, avoiding cycles. This process continues until all cells are connected.

DFS (Depth-First Search)

- Approach: DFS explores as far as possible along each branch before backtracking. It starts from the entrance of the maze and explores paths until it reaches the exit or a dead end. When it reaches a dead end, it backtracks to the most recent junction and explores another path.
- Strategy: DFS is well-suited for maze solving because it tends to follow a single path deeply into the maze, potentially leading to the exit more quickly if it's nearby.
- Memory Usage: DFS typically uses less memory compared to BFS because it explores one path at a time, maintaining only a stack of vertices to be visited.
- Completeness: DFS is not guaranteed to find the shortest path to the exit, but it will eventually find a path if one exists.

BFS (Breadth-First Search)

- Approach: BFS explores all neighbor vertices of the current vertex before moving on to the next level of neighbors. It starts from the entrance of the maze and explores all possible paths level by level until it finds the exit.
- Strategy: BFS is also suitable for maze solving because it systematically explores all possible paths from the entrance to the exit, ensuring that the shortest path is found.
- Memory Usage: BFS typically uses more memory compared to DFS because it maintains a queue of vertices to be visited, which can grow significantly in size, especially for large mazes.
- Completeness: BFS is guaranteed to find the shortest path to the exit, making it an ideal choice if finding the shortest path is a priority.

Here's a summary of how an array-based disjoint set class can be used to generate a maze that contains a path from an entrance to an exit:

1. Initialization: Start by initializing an array-based disjoint set with each cell in the maze as a separate set. Find an example of the 25 disjoint cells of a 5x5 maze in **Figure 3**. Each cell represents a vertex in the maze graph. In **Figure 4**, we assume the entrance is at cell 0 and the exit is at cell 24. We will always generate a maze such that the entrance is always at the top-left cell and the exit is at the bottom-right cell.

2. Generate the Maze: As you construct the maze, you'll merge sets together to create paths. For example, when using Kruskal's algorithm, you'll randomly select edges (representing walls between cells) and check if the cells they connect are in different sets. If they are, you'll perform a union operation to merge the sets. We will assume all cells are connected to their neighbors.

Also, since the weights of the connected cells are equal, they will not be considered. Because all edge weights are equal, any maze generated will be a minimum spanning tree such that all cells can be reached from any given cell in the maze. Now we can use the edges that make up the minimum spanning tree to carve the paths throughout the maze. Remember, these are the edges/walls that are to be removed so that all the cells are connected without a circuit.

{0} {1} {2} {3} {4} {5} {6} {7} {8} {9} {10} {11} {12} {13} {14} {15} {16} {17} {18} {19} {20} {21} {22} {23} {24}

Figure 3

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24

Figure 4

Pseudocode for the `DisjointSet` data structure:

```
class DisjointSet:

    --Initialize the disjoint set with size n

    function initialize(n):

        parent = new array of size n

        rank = new array of size n

        for i from 0 to n-1

            makeSet(i)     --Each set/cell is its own parent initially

    --Create a new set with a single cell

    function makeSet(x):

        parent[x] = -1   --Set x as its own parent; -1 means no parent

        rank[x] = 0     --Set initial rank to 0; height is used as rank

    --Find the representative/root parent of the set containing cell x

    function find(x):

        if parent[x] < 0 then

            return x

        else

            parent[x] = find(parent[x])  --Path compression
```

```
        return parent[x]
    --Merge the sets containing elements x and y; union by rank/height
    function union(x, y):
        xRoot = find(x)
        yRoot = find(y)
        --Union by rank to keep the tree balanced
        if xRoot ≠ yRoot then
            if rank[xRoot] < rank[yRoot] then
                parent[xRoot] = yRoot
            else if rank[xRoot] > rank[yRoot] then
                parent[yRoot] = xRoot
            else
                parent[yRoot] = xRoot
                rank[xRoot] = rank[xRoot] + 1  --one level higher
```

Pseudocode for Kruskal's Algorithm:

```
function KruskalsMazeGenerator(rows, cols):
    --If using ASCII text for the display of the maze to the console, then
    --initialize that rows x cols matrix accordingly here. Let's call it M.
    Initialize M.
    n = rows * cols --Get the number of vertices/cells. For example, n = 20x20 = 400 cells.
    disjointSet = new DisjointSet(n)  --Initialize a disjoint set data structure
    mst = []  --Initialize an empty list/vector to store the minimum spanning tree edges
    --Initialize an empty list/vector to store edges; an edge has a starting vertex (source),
    --and an ending vertex (destination)
    --Use an Edge object/pair/tuple to store those two characteristics.
    edges = []
    --Create an array of edges
    k = 0
    for i = 0 to n-1
        for j = 0 to n-1
            --add the edge between i and j to the edges array if cell i and cell j are neighbors.
            if i and j are neighbors then
                Compute value for cell_i and cell_j.
                edges[k] = (cell_i, cell_j)  --in this case, we have an edge between these two cells
                k = k + 1
    shuffle(edges)  --simulate a random maze generation, i.e., allow random edges to be selected.
```

```
    while mst.length < rows*cols-1 do  --visit all of the cells
        (u, v) = remove an edge from edges
        if disjointSet.find(u) ≠ disjointSet.find(v) then   --if adding this edge doesn't form a cycle
            mst.add((u, v))  --Add the edge to the minimum spanning tree; remove this wall
            Modify M by removing the wall connecting cell u and cell v.
            disjointSet.union(u, v)  --Merge the sets containing u and v
    return M
function print_maze(M):
    for row in M
        print(row)
```

Pseudocode for Driver:

```
function main():
    rows = 4
    cols = 4
    maze = KruskalsMazeGenerator(rows, cols)
    print_maze(maze)
    if solve(maze) then   --use depth-first search or breadth-first search
        print_maze(maze)
    else
        print("Not solvable.")
```

Now answer a few questions:

1) How many walls separate the cells of an NxN grid? **[1 point]**
2) Given an NxN maze with no cycles (a perfect maze), how many walls separate the cells? **[1 point]**
3) What is the runtime complexity of the `find` operation of the disjoint set data structure? **[1 point]**
4) Intuitively, without a detailed analysis, how might we generalize the runtime complexity of the `find` operation. *Hint*: How are the cells stored? **[1 point]**
5) Why do the paths created when generating a maze represent a minimum spanning tree? **[1 point]**